

Meeting Intelligence and Automated Reporting Platform

1. Vision

The Meeting Intelligence and Automated Reporting Platform will capture recurring team meetings, associate them with the operational state of the organization, submit the resulting context for AI analysis, and publish standardized reports into Confluence.

The platform will transform distributed meeting recordings and project-system data into traceable, reusable knowledge artifacts.

The system will support:

- five delivery teams
- team-specific recurring ceremonies
- organization-wide recurring meetings
- individual meeting analysis
- multi-meeting daily aggregation
- Jira sprint-state snapshots
- SharePoint artifact synchronization
- Amazon Transcribe
- Confluence-mediated AI processing
- structured JSON results
- templated Confluence publication

The architecture will be event-driven, idempotent, and safe to reprocess. Each service should be able to receive the same event repeatedly without creating incorrect duplicates or repeating completed work unnecessarily.

2. Organizational Meeting Model

There are five delivery teams.

Each team has its own recurring:

- daily stand-up
- sprint planning meeting
- sprint review
- backlog refinement meeting

There are also centrally managed meetings, including:

- Scrum of Scrums
- Symposium
- central backlog refinement

Each recurring Zoom meeting is represented as a **meeting series**.

Each actual instance of that series is represented as a **meeting occurrence**.

A meeting occurrence is the central logical object in the system. It eventually contains or references:

- Zoom metadata
- one or more MP4 recordings
- one or more VTT transcripts
- normalized meeting metadata
- the applicable Jira snapshot
- applicable SharePoint artifacts
- source-system identifiers
- AI input pages
- AI output JSON
- rendered reports
- publication status

3. Zoom Metadata Convention

Each Zoom meeting has:

- a title
- a description
- one or more recording artifacts

The description is always formatted as valid key-value data.

Example:

```
scope=team
team=payments
meeting_type=daily-standup
artifact_class=temporary
jira_project=PAY
transcription_vocabulary=payments
analysis_profile=daily-standup
```

This metadata is configuration, not merely descriptive text.

It determines:

- the logical meeting type
- the owning team
- whether the meeting is team-scoped or central
- the S3 destination
- the applicable retention policy
- the Amazon Transcribe vocabulary
- the Jira context to associate
- the SharePoint artifacts to synchronize
- whether the direct or aggregation pipeline applies
- the AI analysis profile
- the Confluence labels
- the final report template

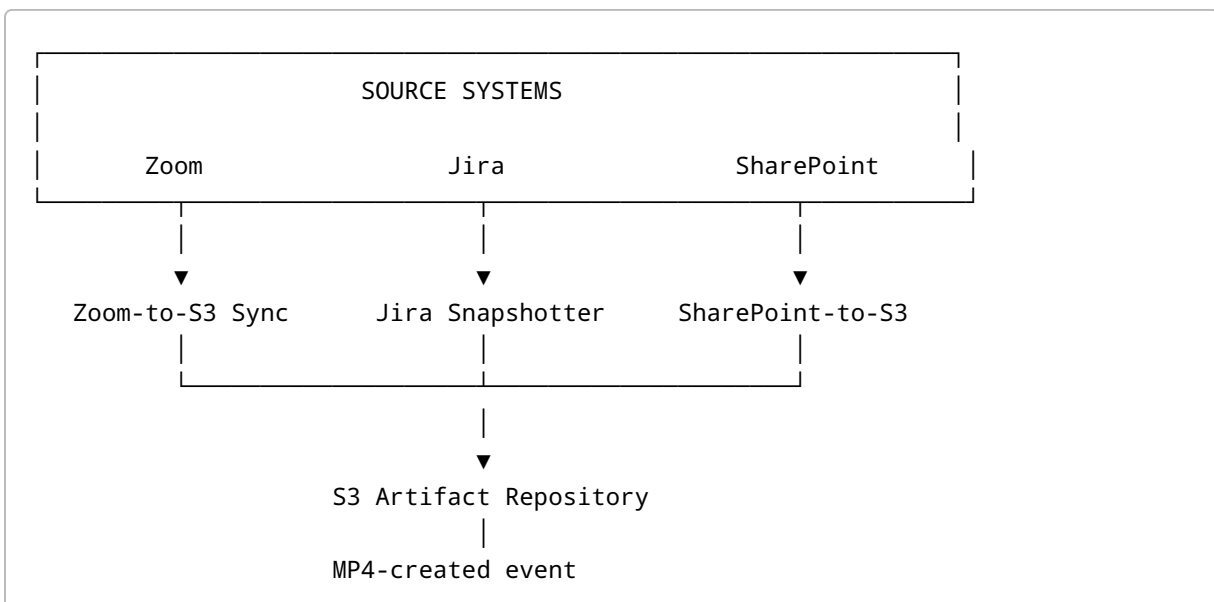
The raw key-value data should be retained, but the system should also validate and normalize it against a defined schema.

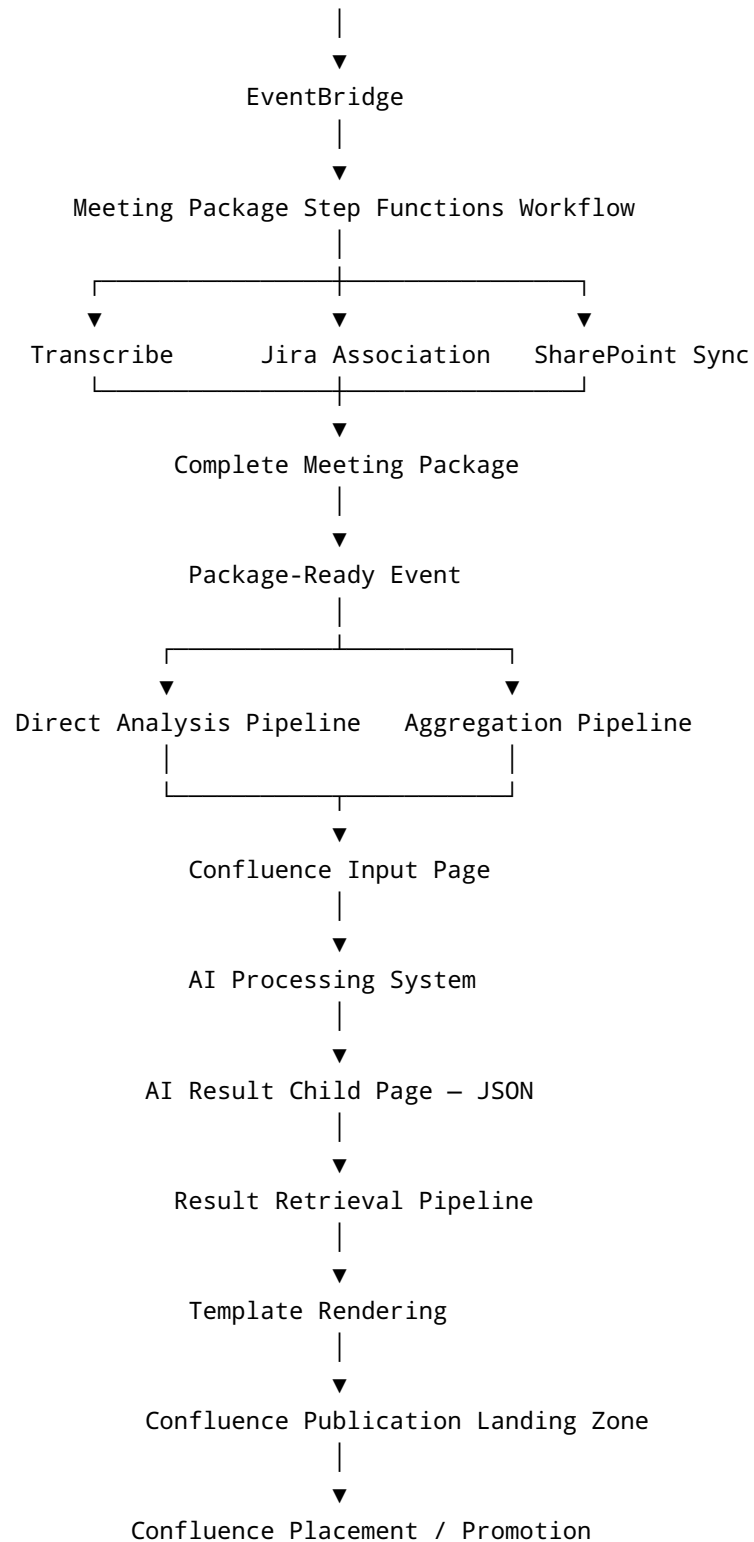
Invalid or incomplete metadata should place the occurrence into an exception queue rather than silently constructing an incorrect path or selecting the wrong analysis workflow.

4. Architectural Overview

The system has four major stages:

1. Source synchronization
2. Meeting-package construction
3. AI analysis
4. Report rendering and publication





5. Source Synchronization

5.1 Zoom-to-S3

Zoom-to-S3 is a scheduled synchronization service.

It repeatedly queries Zoom for completed meeting recordings that are available for download.

Its responsibilities are to:

1. Discover completed recurring-meeting occurrences.
2. Parse the meeting description.
3. Validate the key-value metadata.
4. Download the required MP4 recordings.
5. Reject or mark false-start recordings below a configurable duration or size.
6. Preserve split recording segments when Zoom creates more than one MP4.
7. calculate a deterministic meeting-occurrence identifier.
8. Calculate deterministic S3 object paths.
9. Upload each MP4.
10. Create or update the occurrence manifest.
11. Avoid redownloading an identical recording already present in S3.

Zoom-to-S3 should not directly run the complete downstream pipeline. Its responsibility ends when it has made a valid source artifact available at its deterministic S3 location.

The synchronization itself can be initiated by EventBridge Scheduler. AWS recommends EventBridge Scheduler for managed recurring and one-time schedules, independently of event-bus rules.

5.2 Jira Snapshotter

The Jira Snapshotter is an independent scheduled service.

It periodically captures the state of each team's current sprint and writes timestamped snapshots to S3.

A snapshot should include:

- the raw Jira response
- normalized tickets and relationships
- active sprint identity
- sprint goal
- ticket status
- assignees
- estimates
- blockers
- dependencies
- parent-child relationships
- scope changes

- calculated sprint metrics

The system should preserve a baseline representing the starting state of each sprint.

Derived analytics may include:

- starting scope
- current scope
- completed scope
- remaining scope
- added scope
- removed scope
- blocked work
- reopened work
- unestimated work
- changes since the preceding snapshot
- changes since sprint start

A meeting should normally be associated with the latest valid snapshot whose timestamp is at or before the meeting's effective start time.

5.3 SharePoint-to-S3

SharePoint-to-S3 manages permanent supporting artifacts.

It may run scheduled background synchronization, but the meeting-package workflow must also be able to request synchronization of the specific SharePoint items required for a meeting.

The sync operation should be idempotent:

- If the required current version is already in S3, return the existing reference.
- If the item has changed, synchronize the newer version.
- If the item is missing or inaccessible, return an explicit status.

The S3 representation should include stable SharePoint identifiers such as:

- site identifier
- drive identifier
- item identifier
- item version
- source URL
- synchronization timestamp

SharePoint synchronization is supporting enrichment rather than the primary workflow coordinator.

6. S3 Storage Model

S3 is the canonical artifact repository and the source of truth for package readiness.

A conceptual path model is:

```
s3://meeting-intelligence/  
  source/  
    zoom/  
      {business-date}/  
        {meeting-type}/  
          {scope-or-team}/  
            {occurrence-id}/  
              recording-001.mp4  
              recording-002.mp4  
              source-manifest.json  
  
  transcripts/  
    {business-date}/  
      {occurrence-id}/  
        recording-001.vtt  
        recording-002.vtt  
        transcription-manifest.json  
  
  jira/  
    {team-id}/  
      {sprint-id}/  
        baseline/  
          snapshot.json  
        snapshots/  
          {snapshot-timestamp}/  
            raw.json  
            normalized.json  
            analytics.json  
  
  sharepoint/  
    {logical-domain}/  
      {stable-item-id}/  
        {version}/  
          artifact  
          metadata.json  
  
  packages/  
    {business-date}/  
      {meeting-type}/  
        {scope-or-team}/
```

```
        {occurrence-id}/
        meeting-package.json

analysis/
  requests/
  results/
  rendered/
```

Temporary and permanent artifacts may use separate buckets or separate prefixes. The logical model should not depend on that deployment choice.

S3 paths should be:

- deterministic
- based on normalized metadata
- safe for retries
- stable across repeated synchronization
- independent of free-form meeting titles

S3 tags should be reserved for compact routing, lifecycle, security, and operational attributes. Complete metadata should live in versioned JSON manifests.

7. Event and Orchestration Model

The system should use three AWS mechanisms for three different purposes.

EventBridge: domain event routing

EventBridge receives and routes significant state changes, including:

- recording discovered
- MP4 committed
- package completed
- analysis requested
- AI result discovered
- report rendered
- publication completed
- processing failed

Amazon S3 can send its events to EventBridge, and EventBridge can start a Step Functions execution in response.

Step Functions: bounded workflow orchestration

Step Functions coordinates processes that have a defined beginning, a known series of required operations, and a final success or failure state.

It should be used for:

- meeting-package construction
- direct analysis submission
- AI result retrieval and validation
- template rendering and publication

Step Functions is preferable to a long chain of Lambdas because it makes workflow state, retries, branching, error handling, and execution history visible. AWS specifically positions Step Functions as the coordinator for workflows composed of separate Lambda tasks.

SQS: durable work queues

SQS should sit between asynchronous producers and consumers where work must be buffered, retried, rate-limited, or independently scaled.

Queues may include:

- package-ready queue
- direct-analysis queue
- aggregate-input queue
- Confluence-submission queue
- AI-result-retrieval queue
- rendering queue
- publication queue
- dead-letter queues

SQS durably preserves work and decouples an upstream Lambda from a slower downstream processor. Lambda can consume SQS directly through an event-source mapping.

The recommended rule is:

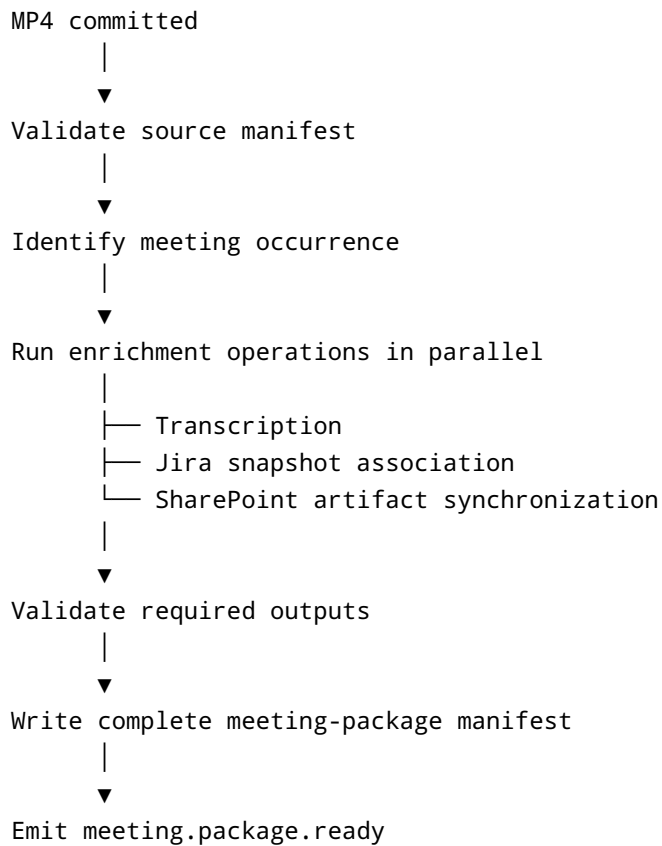
EventBridge announces that something happened, SQS holds work that must be performed, and Step Functions coordinates bounded multi-step operations.

SNS is not required in the initial design. EventBridge provides more useful content-based routing for the system's domain events. SNS could still be introduced later for simple broadcast notifications.

8. Meeting-Package Construction Workflow

When an MP4 is committed to the appropriate S3 source prefix, S3 emits an event through EventBridge.

EventBridge starts the **Meeting Package Construction** state machine.



8.1 Transcription branch

The transcription branch:

1. Checks whether the expected VTT already exists.
2. Reads the transcription profile from normalized metadata.
3. Selects the appropriate vocabulary.
4. Starts Amazon Transcribe when required.
5. waits for completion.
6. Writes the VTT and transcription metadata to deterministic paths.
7. returns the resulting artifact references.

8.2 Jira association branch

The Jira branch:

1. Reads the team, project, sprint, and meeting timestamp.
2. Locates the latest valid snapshot at or before the meeting.
3. Validates that it matches the appropriate team and sprint.
4. Records a stable reference to the snapshot.
5. Optionally copies an immutable snapshot into the meeting package.

8.3 SharePoint branch

The SharePoint branch:

1. Identifies the items required by the meeting metadata.
2. Checks whether their current versions are already synchronized.
3. Synchronizes missing or outdated items.
4. records the S3 and SharePoint references.

8.4 Idempotency

Every branch must be safe to rerun.

A repeated execution should:

- reuse an existing valid VTT
- reuse the already selected Jira snapshot
- reuse current SharePoint copies
- reconstruct the same package identifier
- overwrite only mutable status records
- avoid duplicate Confluence requests

S3 event delivery and queue processing should be treated as at-least-once. Idempotency is therefore a core requirement rather than an optimization.

9. Meeting Package

The completed package manifest is the canonical representation of a meeting occurrence.

Example:

```
{
  "schemaVersion": "1.0",
  "packageId": "2026-07-23-payments-daily-standup-abc123",
  "status": "ready",
  "businessDate": "2026-07-23",
  "meeting": {
    "seriesId": "zoom-series-123",
    "occurrenceId": "zoom-occurrence-456",
    "scope": "team",
    "teamId": "payments",
    "meetingType": "daily-standup",
    "scheduledStart": "2026-07-23T13:00:00Z",
    "actualStart": "2026-07-23T13:02:00Z"
  },
}
```

```

"routing": {
  "pipeline": "aggregation",
  "analysisProfile": "daily-brief-team-input",
  "outputTemplate": "daily-brief"
},
"recordings": [
  {
    "segment": 1,
    "s3Uri": "s3://.../recording-001.mp4",
    "durationSeconds": 932
  }
],
"transcripts": [
  {
    "segment": 1,
    "s3Uri": "s3://.../recording-001.vtt"
  }
],
"jiraSnapshot": {
  "snapshotId": "payments-sprint-42-20260723T125500Z",
  "s3Uri": "s3://.../normalized.json"
},
"sharePointArtifacts": [],
"processing": {
  "packageConstructedAt": "2026-07-23T14:05:00Z"
}
}

```

Multiple recording segments may initially remain as separate MP4 and VTT artifacts.

A later enhancement may introduce transcript consolidation. That should be represented as an additional derived artifact rather than replacing the original transcripts.

10. Pipeline Routing

Once the package is complete, the construction workflow emits:

```
meeting.package.ready
```

The event includes:

- package identifier
- package-manifest location
- business date

- meeting type
- team or central scope
- selected processing pipeline
- analysis profile

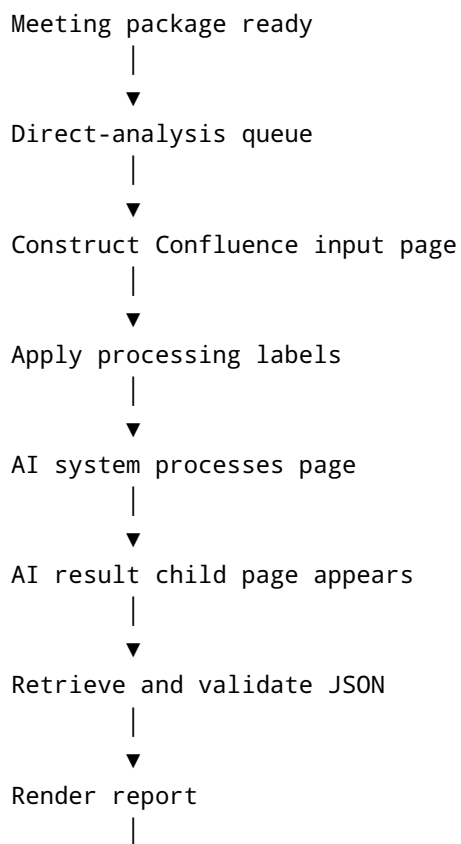
The event is routed into one of two downstream pipelines.

11. Direct Analysis Pipeline

The Direct Analysis Pipeline processes one meeting package independently.

It applies to:

- sprint planning
- sprint review
- team backlog refinement
- central backlog refinement
- Scrum of Scrums
- Symposium
- other individually analysed meetings



▼
Publish to landing zone

A direct request normally contains a single meeting package, but the request schema should accept a list of package references. This keeps it compatible with the Aggregation Pipeline.

12. Aggregation Pipeline

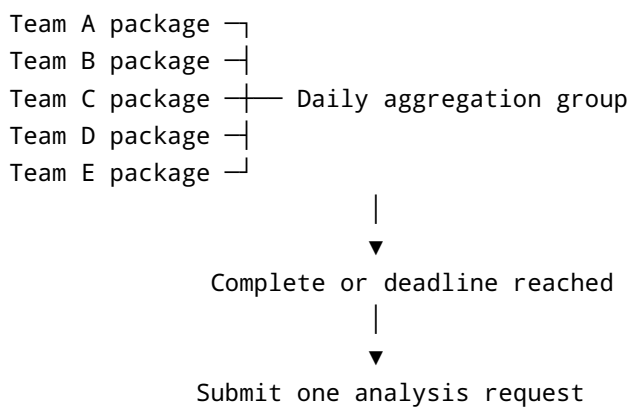
The Aggregation Pipeline waits for a defined collection of meeting packages.

The first use case is the Daily Brief.

Each of the five teams produces one daily stand-up package. Those packages are grouped by:

- business date
- meeting type
- aggregation profile

The Daily Brief aggregation expects one package from each configured team.



12.1 Aggregation state

S3 remains the artifact source of truth, but a small DynamoDB coordination table is recommended for aggregation state.

The table would contain:

- aggregation group identifier
- business date
- expected team identifiers
- received package identifiers
- deadline

- current status
- analysis request identifier
- completion timestamp

Example aggregation identifier:

```
daily-brief#2026-07-23
```

DynamoDB avoids repeatedly listing and scanning S3 every time a package arrives. The actual packages remain in S3; DynamoDB stores only lightweight coordination state.

12.2 Completion rule

The aggregation proceeds when either:

1. all expected team packages are ready; or
2. the configured business-time deadline has passed.

The deadline must use an explicit named time zone, such as `America/Toronto`, rather than relying on an ambiguous abbreviation such as EST.

When the deadline expires, the analysis request should clearly identify:

- received teams
- missing teams
- failed teams
- late teams
- whether the brief is complete or partial

A scheduled deadline event can be created through EventBridge Scheduler, while package-arrival events can update the same aggregation record.

12.3 Aggregation output

The aggregator produces a list of meeting-package inputs rather than physically combining all source artifacts.

```
{
  "analysisRequestId": "daily-brief-2026-07-23",
  "analysisProfile": "daily-brief",
  "businessDate": "2026-07-23",
  "inputs": [
    {"teamId": "team-a", "packageUri": "s3://..."},
    {"teamId": "team-b", "packageUri": "s3://..."},
    {"teamId": "team-c", "packageUri": "s3://..."},
    {"teamId": "team-d", "packageUri": "s3://..."}
  ]
}
```

```
    {"teamId": "team-e", "packageUri": "s3://..."}  
  ]  
}
```

Both pipelines therefore converge on the same analysis-request contract:

An analysis request contains one or more meeting packages and an analysis profile.

13. Confluence-Mediated AI Analysis

Confluence is used as the exchange surface between the AWS pipeline and the AI analysis system.

This process has two distinct page types.

13.1 AI input page

A general-purpose Lambda receives an analysis request and constructs a Confluence page containing the complete analysis input.

The page may contain:

- meeting metadata
- transcript content
- Jira sprint state
- calculated Jira analytics
- SharePoint references or extracted content
- package references
- required analysis instructions
- expected output schema

For a Daily Brief, the page contains the relevant content from all available team stand-ups.

For direct analysis, it usually contains one meeting package.

The page is placed into a dedicated AI-processing landing area and receives machine-readable labels.

Example labels:

```
automation-ai-request  
analysis-daily-brief  
status-ready-for-ai  
request-daily-brief-2026-07-23  
schema-v1
```

The labels determine which Rovo agent or other AI processor handles the page.

Confluence supports adding labels to content and querying pages by label. It also exposes page-child relationships through its REST APIs.

13.2 AI result child page

The AI processor reads the labeled input page and creates a child page underneath it.

That child page contains the AI result, normally as structured JSON.

Example:

```
{
  "schemaVersion": "1.0",
  "analysisType": "daily-brief",
  "executiveSummary": "...",
  "teams": [],
  "crossTeamDependencies": [],
  "blockers": [],
  "risks": [],
  "decisions": [],
  "actionItems": [],
  "followUps": []
}
```

The result child page should also carry labels such as:

```
automation-ai-result
status-ai-complete
parent-request-daily-brief-2026-07-23
schema-v1
```

The JSON schema should be versioned and validated after retrieval.

Confluence is not the permanent machine-readable source of truth for the result. After discovery, the JSON should be copied into S3.

14. AI Result Retrieval

A later asynchronous process checks outstanding AI requests.

Its responsibility is to determine whether the expected result child page exists.

This process may combine:

- a scheduled reconciliation Lambda
- event-driven messages where Confluence integration permits
- delayed SQS retries
- a maximum processing deadline

The reconciler:

1. Searches for outstanding request pages.
2. Reads their direct child pages.
3. Identifies a child carrying the expected result labels.
4. extracts the JSON result.
5. validates the JSON against the required schema.
6. writes the result to S3.
7. changes the request status to complete.
8. emits an `analysis.result.ready` event.

Polling should be considered a reconciliation mechanism, not the entire workflow engine. SQS delayed retries or Step Functions wait states can manage individual outstanding requests, while a periodic reconciler catches missed or orphaned work.

The process must distinguish among:

- not yet processed
- processed successfully
- malformed result
- incorrect schema
- duplicate result
- processing timeout
- permanent failure

15. Report Rendering

The structured JSON result is passed into a deterministic rendering service.

The renderer should not ask the AI to generate final Confluence formatting.

Instead, it should:

1. Load the appropriate template.
2. Validate the result schema.
3. Map JSON fields to template sections.
4. generate Confluence storage-format content or another supported page representation.
5. produce a rendered-page artifact.
6. preserve links to the original meeting packages and AI request.

Templates should live in source control and be deployed with the renderer.

A lightweight JavaScript or TypeScript templating library is suitable. The design should prioritize:

- deterministic output
- unit testing
- local rendering
- version-controlled templates
- schema validation
- safe escaping
- minimal runtime dependencies

The initial implementation should not depend on editable Confluence templates as the sole template source. Source-controlled templates are easier to test, review, version, and deploy consistently.

16. Confluence Publication

The renderer publishes the completed page into a Confluence automation landing zone.

The AWS service does not need to determine the final location in the complete Confluence hierarchy.

Instead, it applies labels that describe:

- report type
- team
- sprint
- business date
- visibility
- publication destination
- lifecycle state
- schema version

Example:

```
automation-publish-ready  
report-daily-brief  
business-date-2026-07-23  
destination-delivery-leadership  
visibility-internal
```

A Confluence-side automation process can then:

- place or copy the page into the appropriate hierarchy
- apply permissions
- expose it to the correct audience

- archive or relabel the landing-zone copy
- mark the publication as complete

This keeps the AWS publication service simple and avoids embedding detailed Confluence information architecture into every Lambda.

17. Domain Events

The platform should use a small, documented set of versioned domain events.

Suggested events include:

```
zoom.recording.discovered
zoom.recording.committed
meeting.package.started
meeting.transcription.completed
meeting.jira-associated
meeting.sharepoint-associated
meeting.package.ready
aggregation.input.received
aggregation.ready
analysis.request.created
analysis.confluence-page.created
analysis.result.discovered
analysis.result.ready
report.rendered
report.published
processing.failed
```

A standard event envelope should contain:

```
{
  "eventId": "uuid",
  "eventType": "meeting.package.ready",
  "eventVersion": "1.0",
  "occurredAt": "2026-07-23T14:05:00Z",
  "correlationId": "meeting-occurrence-id",
  "causationId": "preceding-event-id",
  "payload": {}
}
```

The correlation identifier makes it possible to follow one meeting occurrence or aggregate request through the complete system.

18. Reliability and Recovery

Every asynchronous queue should have:

- a dead-letter queue
- an explicit retry policy
- an alarm on message age
- an alarm on dead-letter volume
- a documented replay process

A failed operation should preserve:

- input event
- correlation identifier
- failure category
- retry count
- service name
- execution identifier
- last error
- next permitted action

Reprocessing should be supported at several levels:

- rediscover one Zoom occurrence
- replay one MP4-created event
- rerun package construction
- reassociate the Jira snapshot
- resynchronize SharePoint
- resubmit analysis
- retrieve an existing AI result
- rerender a report
- republish a page

Lambda asynchronous failures and SQS messages can be retained in dead-letter queues for later reprocessing.

19. Implementation Direction

The default implementation language should be TypeScript running on a currently supported Node.js Lambda runtime.

This provides:

- a lightweight deployment model
- strong JSON and schema tooling
- good AWS SDK support

- straightforward templating
- shared types between events, manifests, and Lambdas
- manageable local testing

Infrastructure should be managed through Terraform.

Infrastructure should be divided by bounded system area rather than by individual function.

Potential Terraform stacks or modules include:

- core event infrastructure
- Zoom synchronization
- meeting-package construction
- Jira snapshotting
- SharePoint synchronization
- direct analysis
- aggregation
- Confluence AI exchange
- result retrieval
- report rendering
- publication
- monitoring and alarms

Shared Terraform modules should define standard:

- Lambda configuration
- SQS queues and dead-letter queues
- EventBridge rules
- Step Functions logging
- IAM boundaries
- CloudWatch alarms
- encryption
- tagging

20. Recommended AWS Service Boundaries

Requirement	Recommended service
Recurring Zoom discovery	EventBridge Scheduler and Lambda
Recurring Jira snapshots	EventBridge Scheduler and Lambda
S3 event routing	S3 to EventBridge
Package construction	Step Functions Standard Workflow
Parallel enrichment	Step Functions Parallel state

Requirement	Recommended service
Durable work buffering	SQS
Domain event routing	EventBridge
Aggregate coordination	DynamoDB
Artifact source of truth	S3
Lightweight transformation	Lambda
Long-running external completion	Step Functions wait/retry plus reconciliation
Operational monitoring	CloudWatch
Secrets	Secrets Manager
Infrastructure deployment	Terraform

A Standard Step Functions workflow is the safer default for package construction because external transcription and synchronization operations may be long-running and require durable execution history. Express workflows are intended for shorter, high-volume processing and use an at-least-once execution model.

21. Key Design Principles

Deterministic identity

Every series, occurrence, source artifact, package, analysis request, and report receives a stable identifier.

Idempotency

Every service may safely receive the same request more than once.

Artifact immutability

Source recordings, transcripts, snapshots, and retrieved AI results should normally be immutable and versioned.

Explicit state

Workflow status should be represented in manifests, state-machine executions, queue messages, or coordination records—not inferred only from Lambda logs.

Loose coupling

Services communicate through events, queues, manifests, and stable schemas.

Structured AI output

AI analysis produces versioned JSON before any final page is rendered.

Confluence as exchange and publication surface

Confluence provides labeled processing pages and user-facing reports, while S3 retains canonical machine-readable artifacts.

Recoverability

Every important stage can be independently replayed.

Generic analysis contract

Direct and aggregate processing use the same request structure: an analysis profile plus one or more meeting packages.

22. Initial Delivery Phases

Phase 1 — Capture and package construction

Deliver:

- Zoom metadata schema
- Zoom-to-S3 synchronization
- deterministic S3 structure
- Jira Snapshotter
- transcription flow
- Jira association
- SharePoint synchronization
- package-construction state machine
- package-ready events

Phase 2 — Direct analysis

Deliver:

- direct-analysis queue
- Confluence input-page construction
- labels and analysis routing
- AI result child-page convention
- result retrieval
- JSON validation
- one report renderer
- Confluence landing-zone publication

Phase 3 — Daily Brief aggregation

Deliver:

- team registry
- DynamoDB aggregation state
- arrival tracking
- deadline handling
- partial-brief rules
- Daily Brief input construction
- Daily Brief JSON schema
- Daily Brief renderer

Phase 4 — Operational hardening

Deliver:

- complete dead-letter and replay processes
- alarms and dashboards
- execution correlation
- administrative reprocessing tools
- schema migration handling
- load and failure testing
- security and permission review

Phase 5 — Extended intelligence

Potential additions include:

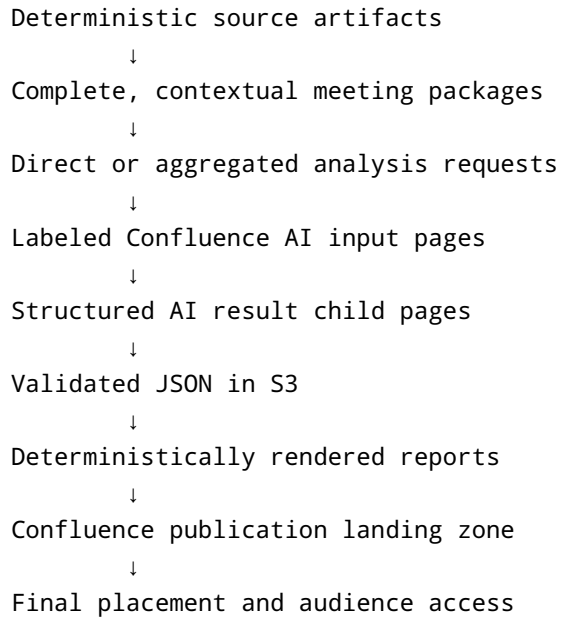
- consolidation of split VTT transcripts
- weekly or sprint-level aggregation
- historical trend analysis
- semantic indexing
- cross-meeting action-item tracking
- automated Jira updates
- report quality evaluation
- human approval before publication

23. Target Outcome

The completed platform will provide a reliable event-driven chain from meeting capture to published organizational knowledge:

Recurring meetings

↓



The architecture allows the system to grow without turning every new meeting type into a custom end-to-end microservice.

New use cases should generally require only:

- a metadata configuration
- an analysis profile
- an output JSON schema
- a rendering template
- routing labels

The common ingestion, package-construction, AI exchange, retrieval, rendering, and publication infrastructure remains shared.